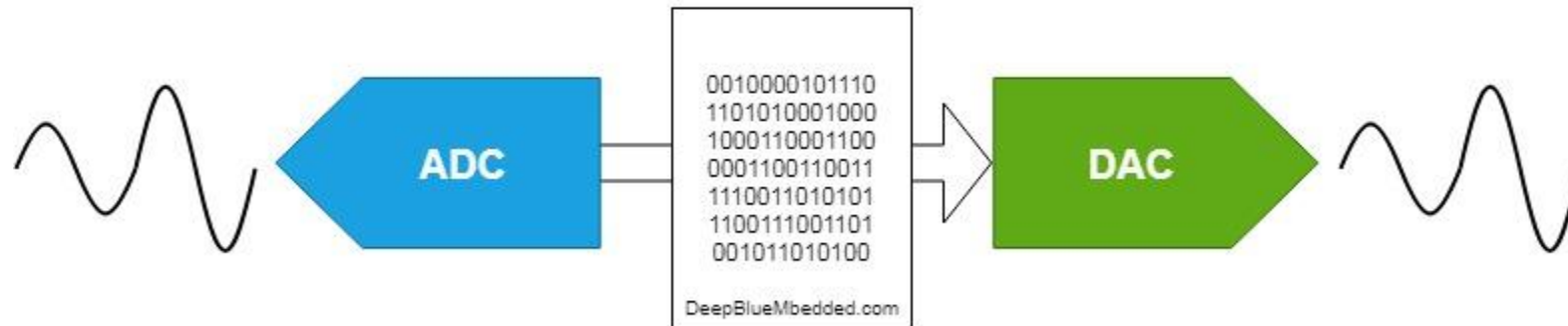
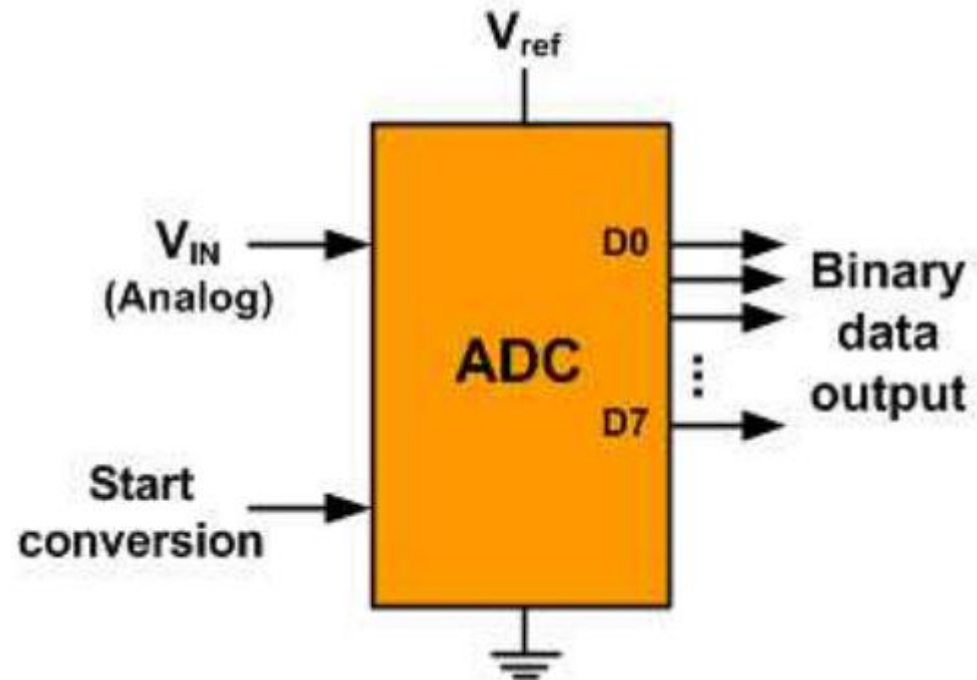
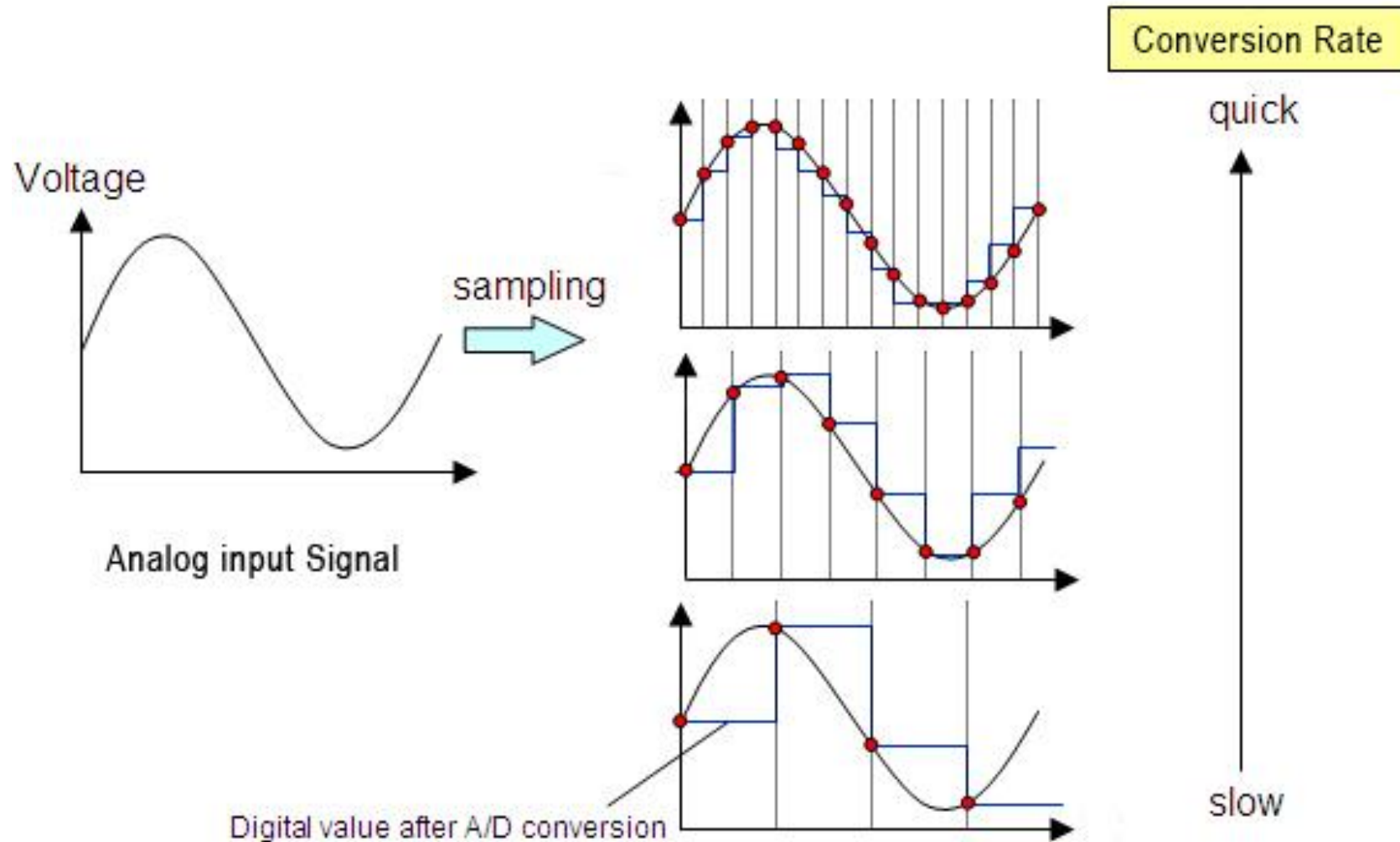


Conversor A/D





Frequência de amostragem



Resolução (em volts)

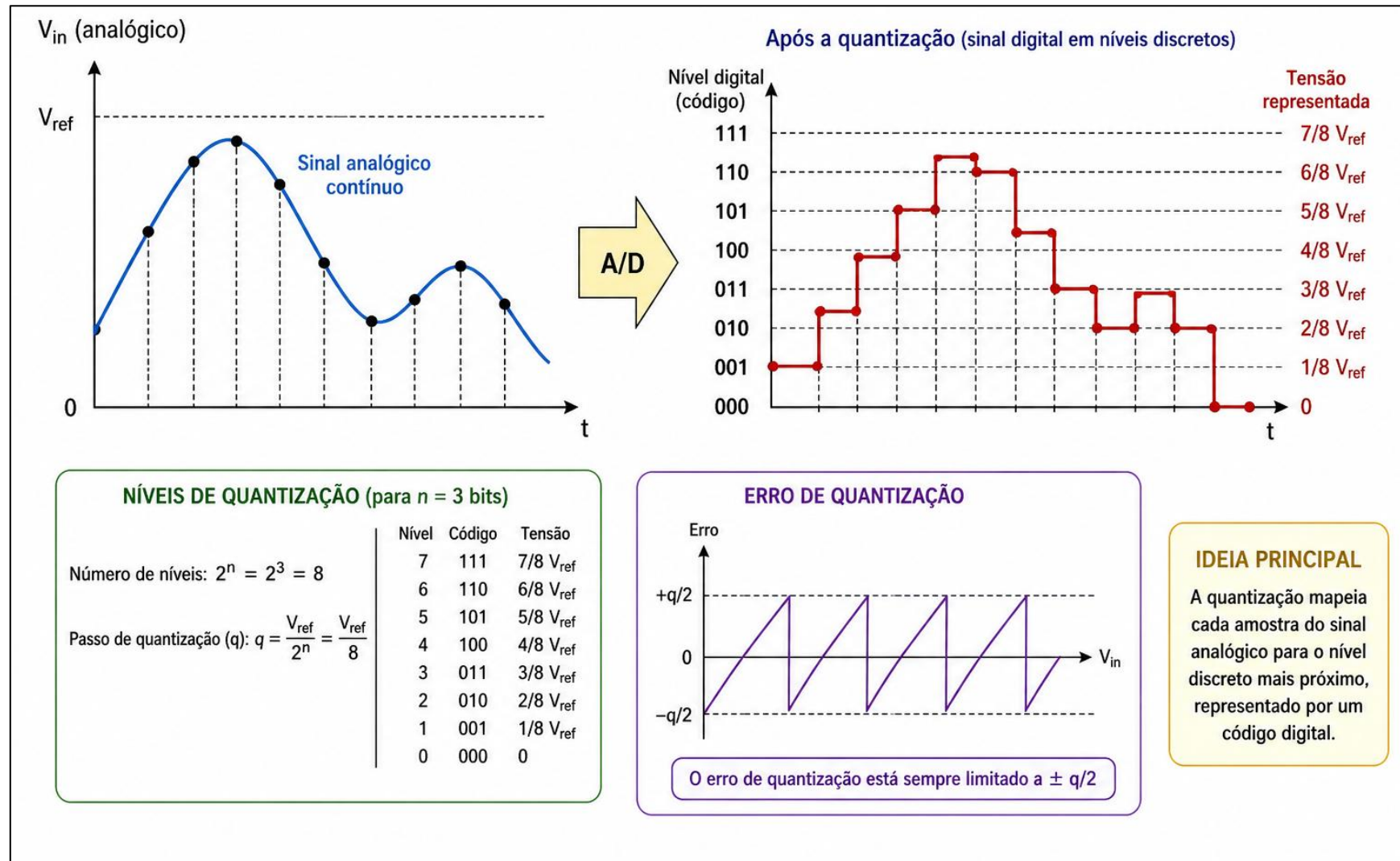
$$Resolution(Q) = \frac{FSR}{2^n}$$

FSR = Full-Scale Range

n-bit	Number of steps	Step size
8	256	5V /256 = 19.53 mV
10	1024	5V /1024 = 4.88 mV
12	4096	5V /4096 = 1.2 mV
16	65,536	5V /65,536 = 0.076 mV
Note: $V_{ref} = 5V$		

Table 13-1: Resolution versus Step Size for ADC ($V_{ref} = 5V$)

Quantização em um conversor A/D



V_{ref}

V _{ref} (V)	V _{in} in Range (V)	Step Size (mV)
5.00	0 to 5	5 / 256 = 19.53
4.00	0 to 4	4 / 256 = 15.62
3.00	0 to 3	3 / 256 = 11.71
2.56	0 to 2.56	2.56 / 256 = 10
2.00	0 to 2	2 / 256 = 7.81
1.28	0 to 1.28	1.28 / 256 = 5
1.00	0 to 1	1 / 256 = 3.90
Note: In an 8-bit ADC, step size is $V_{ref}/256$		

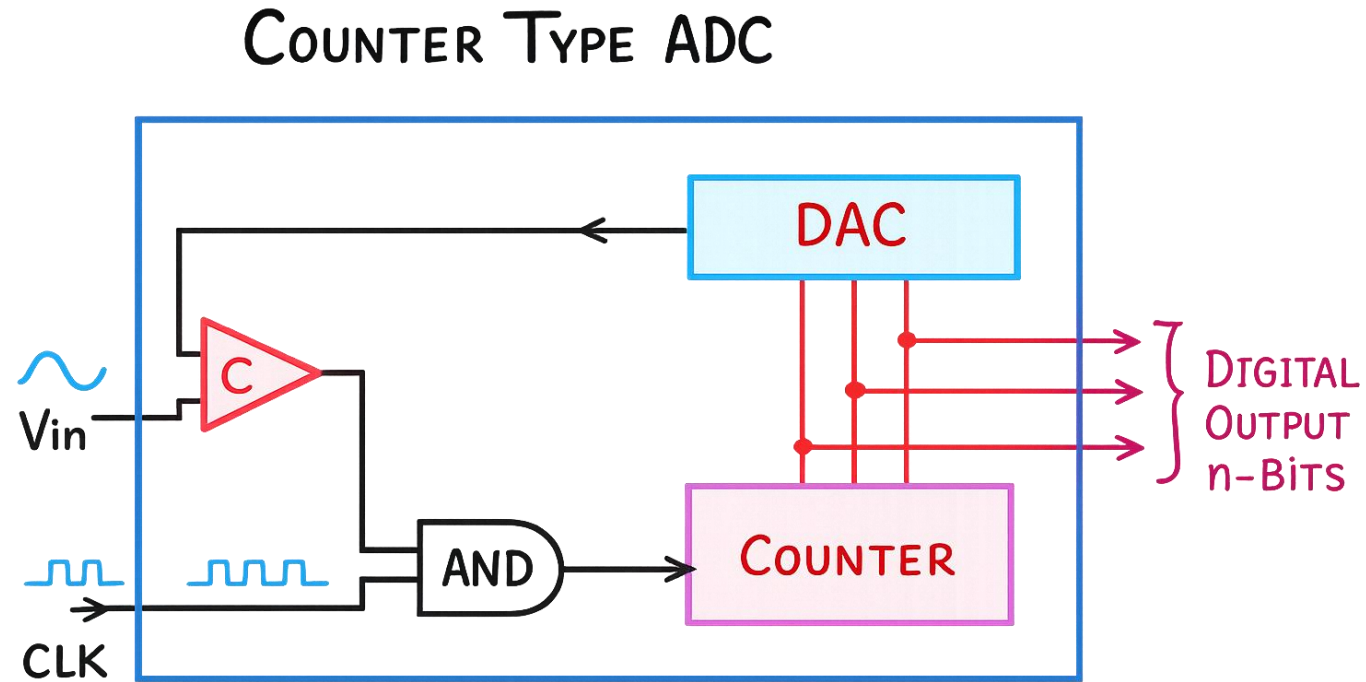
Table 13-2: V_{ref} Relation to V_{in} Range for an 8-bit ADC

V _{ref} (V)	V _{in} Range (V)	Step Size (mV)
5.00	0 to 5	5 / 1024 = 4.88
4.96	0 to 4.096	4.096 / 1024 = 4
3.00	0 to 3	3 / 1024 = 2.93
2.56	0 to 2.56	2.56 / 1024 = 2.5
2.00	0 to 2	2 / 1024 = 2
1.28	0 to 1.28	1.28 / 1024 = 1.25
1.024	0 to 1.024	1.024 / 1024 = 1
Note: In a 10-bit ADC, step size is $V_{ref}/1024$		

Table 13-3: V_{ref} Relation to V_{in} Range for an 10-bit ADC

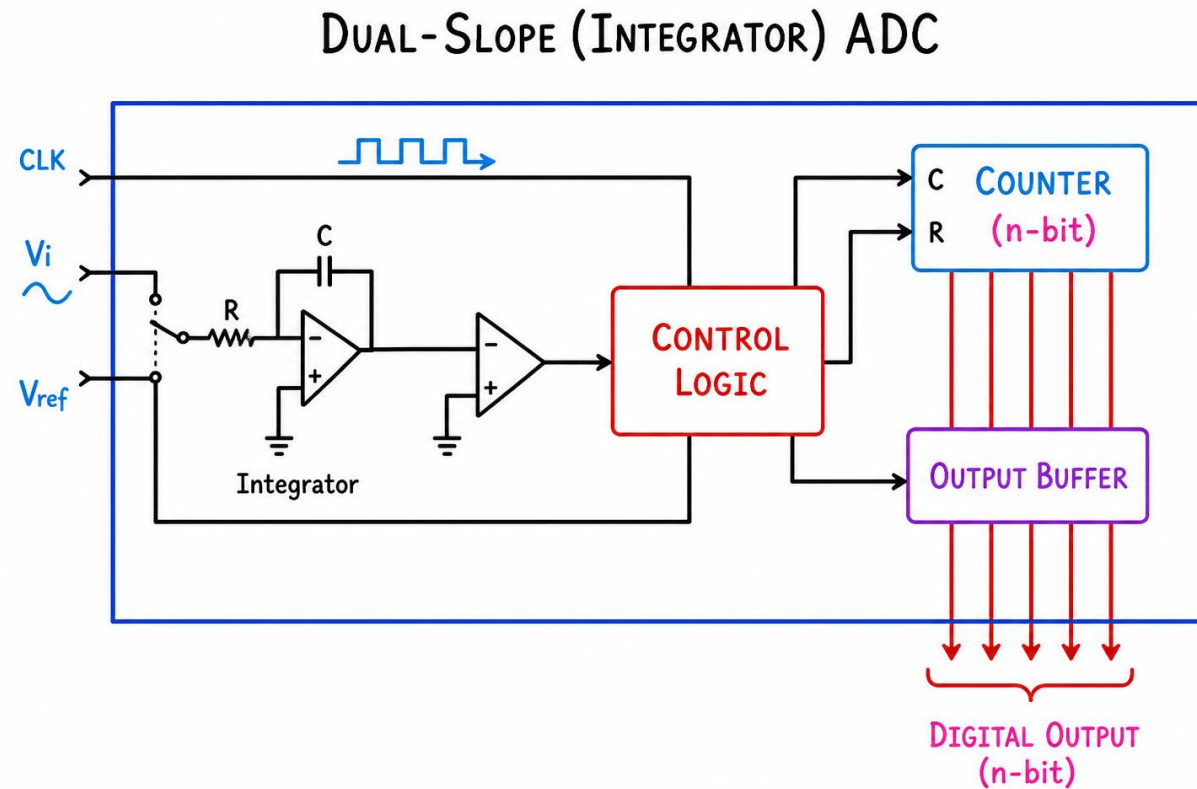
Tipos de conversores A/D

- Tipo contador (single-slope)



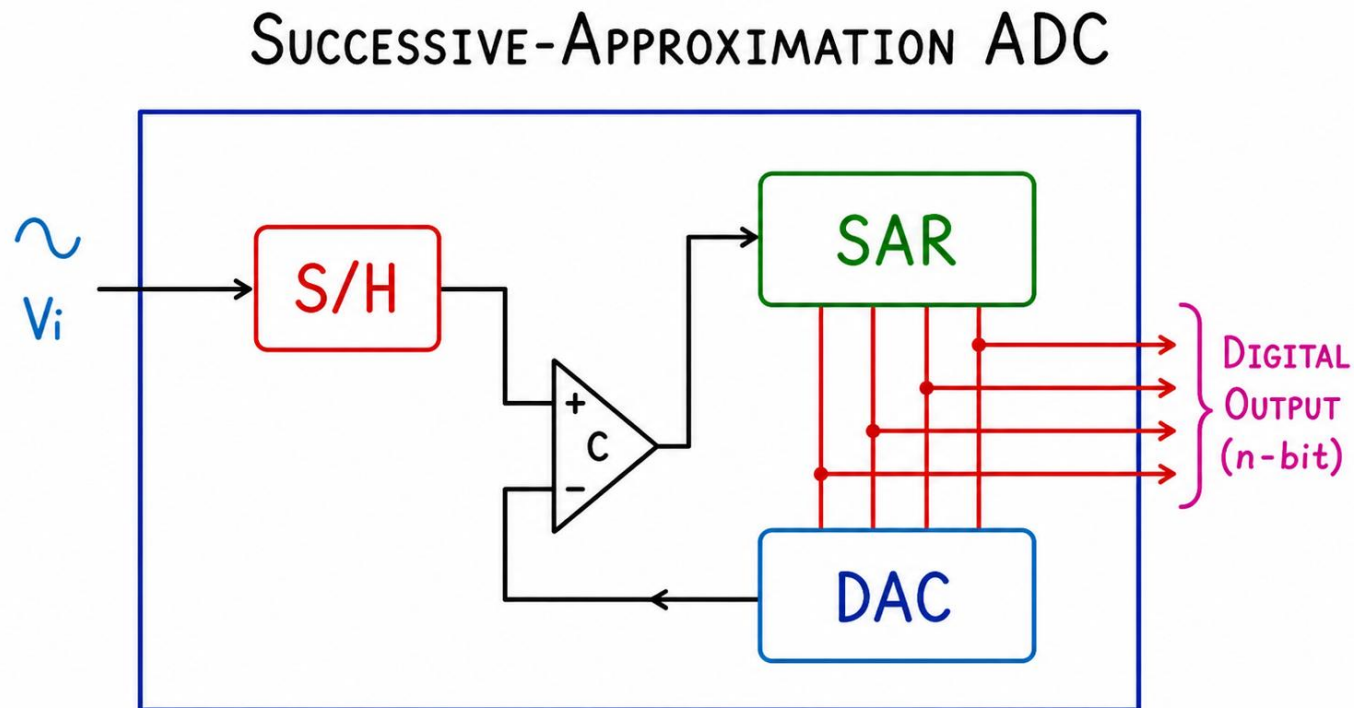
Tipos de conversores A/D

- Do tipo integrador (Dual-Slope)

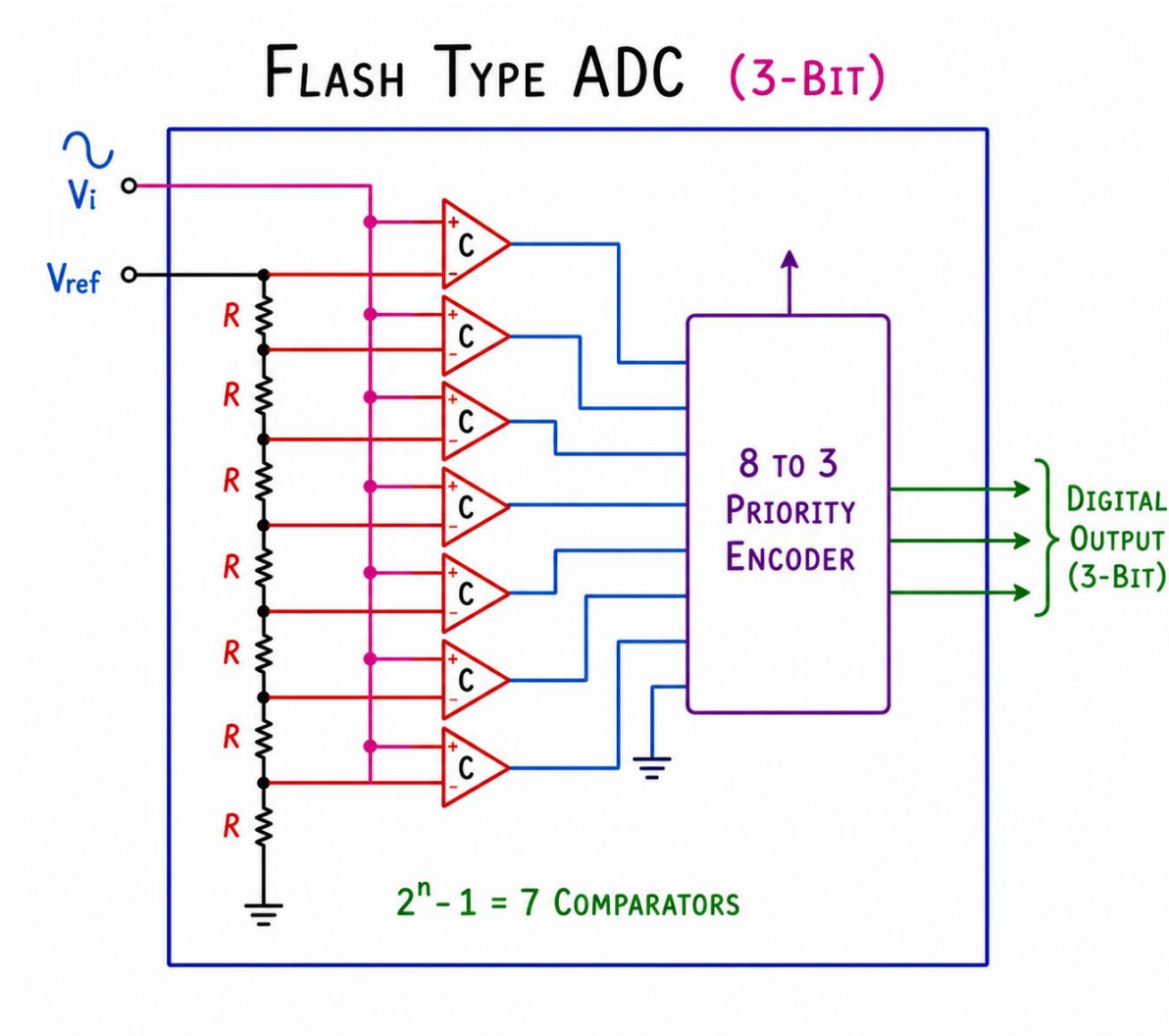


Tipos de conversores A/D

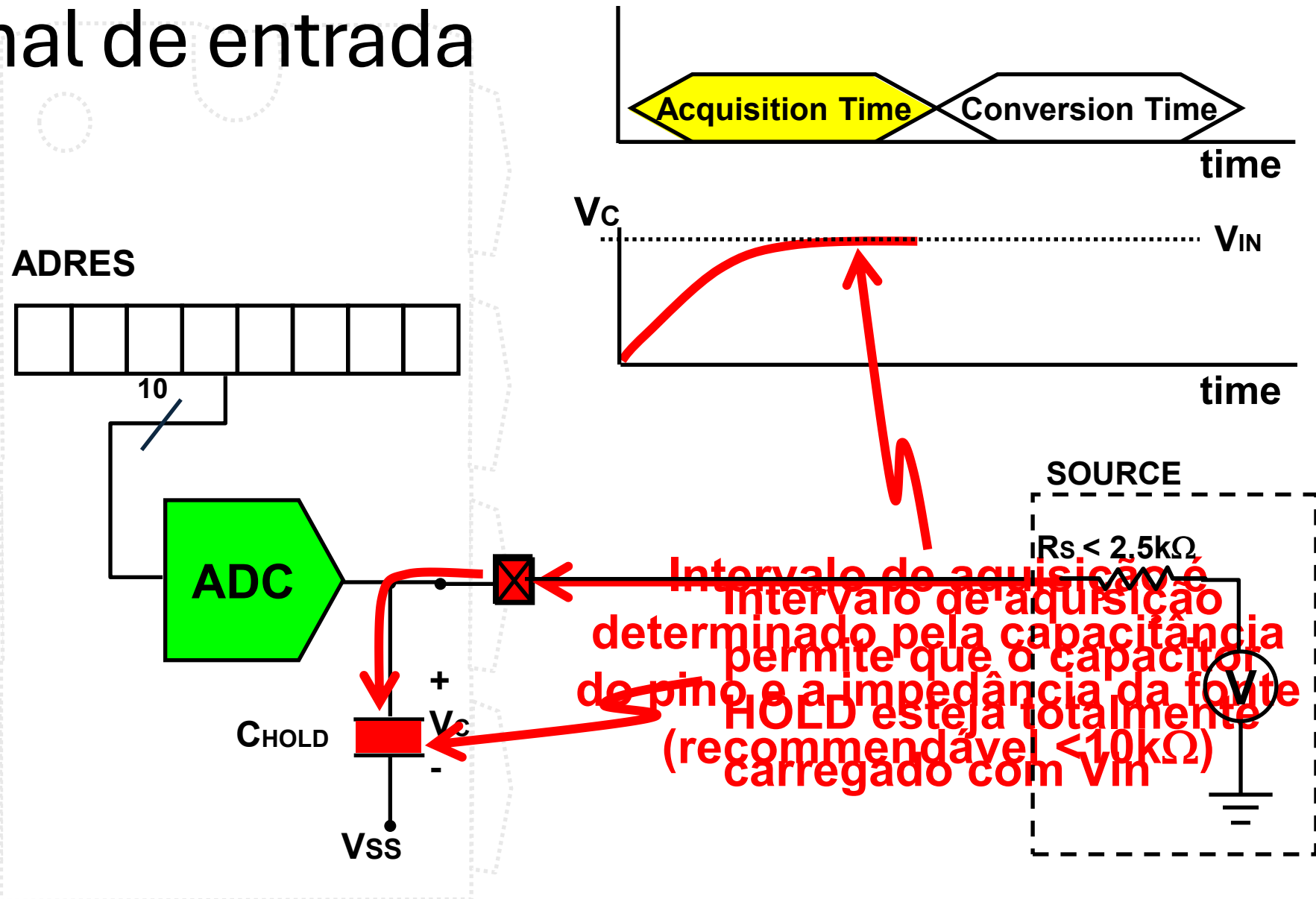
- Do tipo aproximação sucessiva (SAR – Successive Approximation ADC)



- Do tipo flash



Tempo de amostragem: tempo para aquisição do sinal de entrada



Tempo de conversão

ADRES

ADC Result

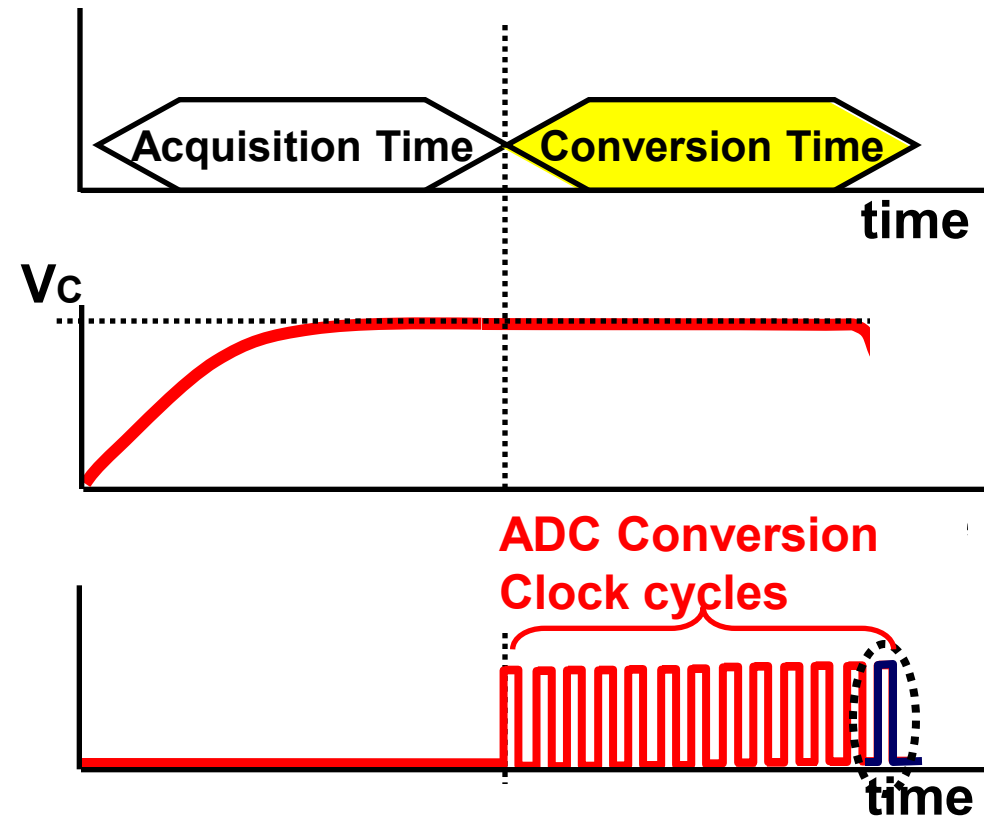
10

ADC

C_{HOLD}

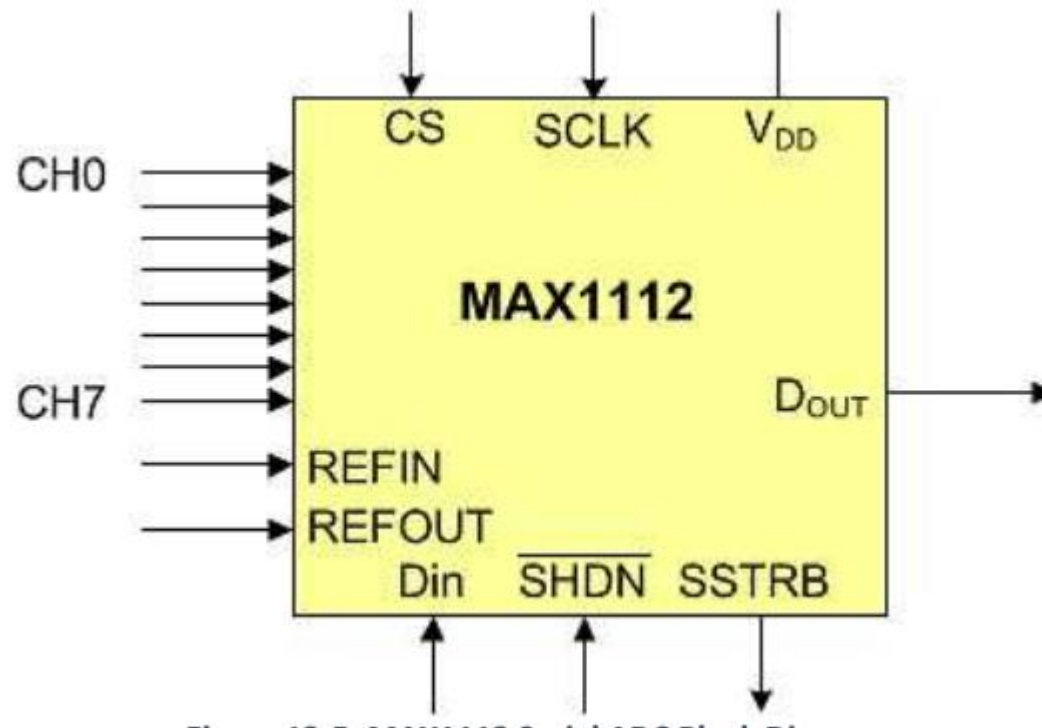
V_{SS}

+
V_C
-

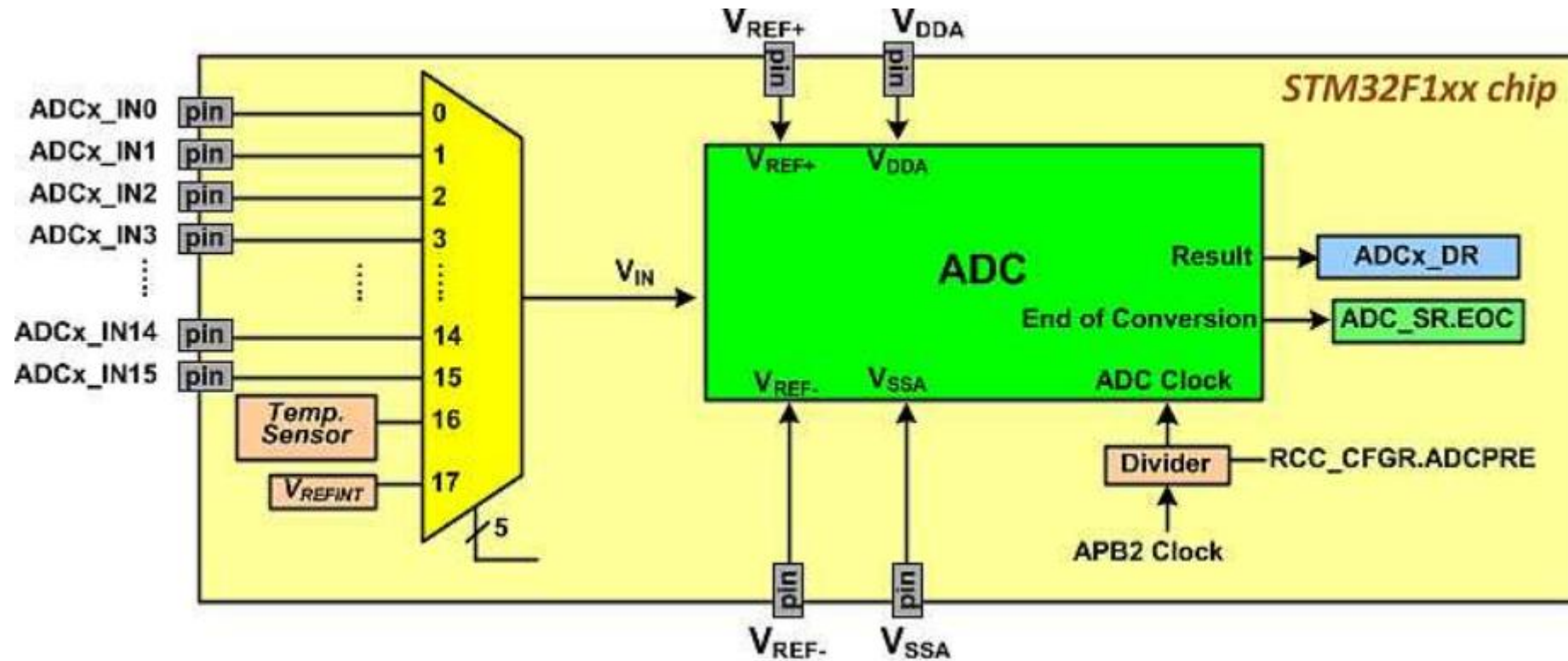


Canais de entrada analógica

- Através do uso de **multiplexadores analógicos**, é possível ter várias entradas analógicas associada a apenas um conversor AD (uma conversão por vez)



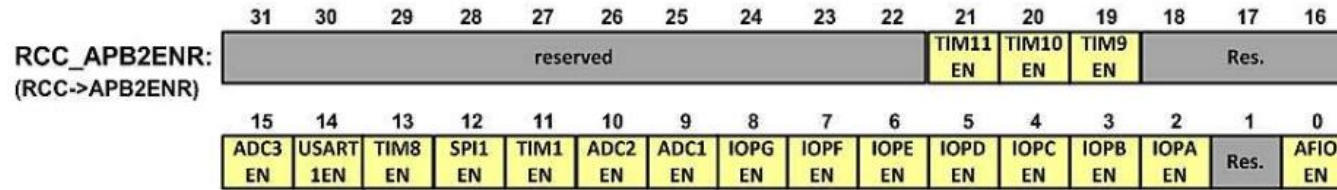
Conversor AD do STM32F103



Registradores associados ao conversor AD

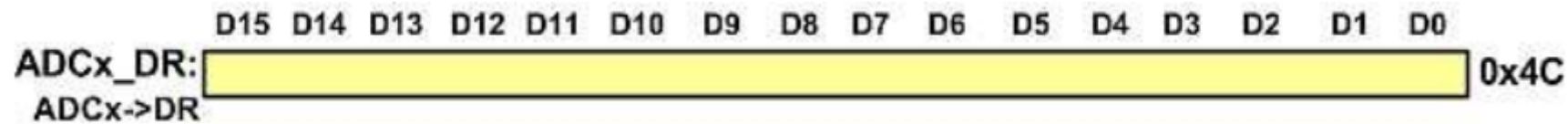
Register name	Offset	Description
ADCx_SR	0x0000	Status Register
ADCx_DR	0x004C	Regular Data Register
ADCx_CR1	0x0004	Control Register 1
ADCx_CR2	0x0008	Control Register 2
ADCx_SMPR1	0x000C	Sample time register 1
ADCx_SMPR2	0x0010	Sample time register 2
ADCx_SQR1	0x002C	Regular Sequence register 1
ADCx_SQR2	0x0030	Regular Sequence register 2
ADCx_SQR3	0x0034	Regular Sequence register 3

- Habilitando o clock para o conversor AD (max. 12MHz)

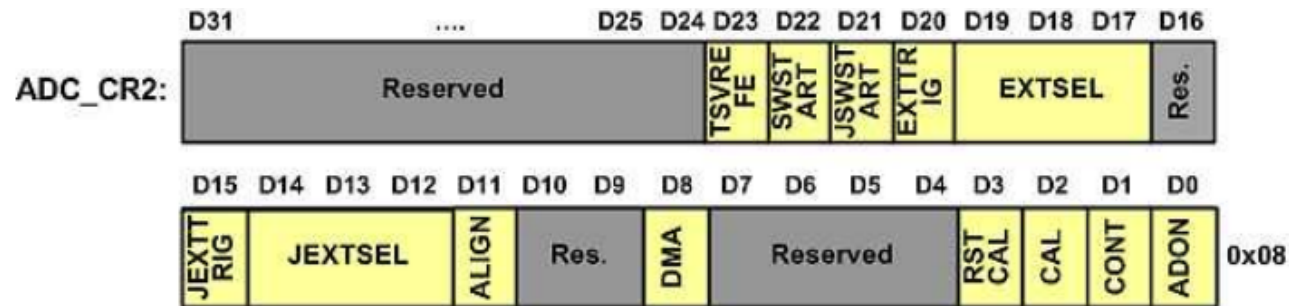


Field	Description										
MCO	Microcontroller Clock Output										
USBPRE	USB Prescaler										
PLLMUL	PLL Multiplication factor										
PLLXTPRE	LSB of division factor PREDIV1										
PLLSRC	PLL entry clock source										
ADCPRE	ADC Prescaler: the bits are used to set the frequency of ADC clock. <table> <tr> <th>ADCPRE value</th><th>ADC Clock Frequency</th></tr> <tr> <td>00</td><td>PCLK2 (APB2 clock) divided by 2</td></tr> <tr> <td>01</td><td>PCLK2 (APB2 clock) divided by 4</td></tr> <tr> <td>10</td><td>PCLK2 (APB2 clock) divided by 6</td></tr> <tr> <td>11</td><td>PCLK2 (APB2 clock) divided by 8</td></tr> </table>	ADCPRE value	ADC Clock Frequency	00	PCLK2 (APB2 clock) divided by 2	01	PCLK2 (APB2 clock) divided by 4	10	PCLK2 (APB2 clock) divided by 6	11	PCLK2 (APB2 clock) divided by 8
ADCPRE value	ADC Clock Frequency										
00	PCLK2 (APB2 clock) divided by 2										
01	PCLK2 (APB2 clock) divided by 4										
10	PCLK2 (APB2 clock) divided by 6										
11	PCLK2 (APB2 clock) divided by 8										
PPRE2	APB2 clock prescaler										
PPRE1	APB1 clock prescaler										
HPRE	AHB clock prescaler										
SWS	System clock Switch Status										
SW	System clock Switch										

- Registro de dados da conversão

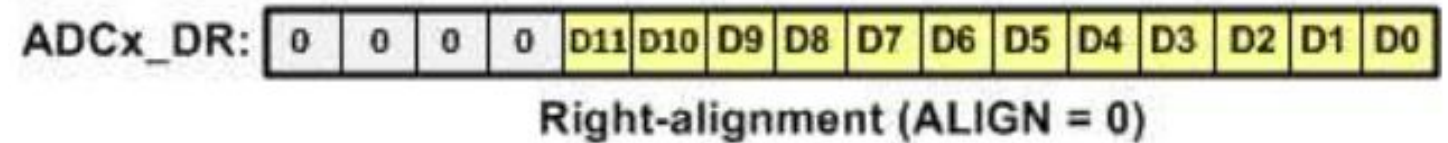
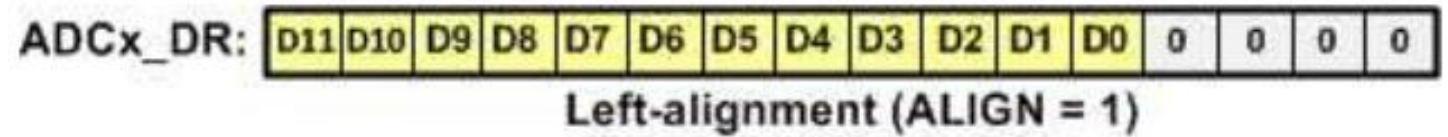


- Registro de controle



Field	Description
ALIGN	Data alignment (0: right alignment, 1: left alignment)
DMA	Direct Memory Access (0: disabled, 1: enabled)
CAL	ADC Calibration To calibrate the ADC, set to bit to 1. The bit clears by hardware, when the calibration finishes.
CONT	Continuous (0: single conversion, 1: continuous conversion)
ADCON	ADC ON/OFF (0: ADC OFF, 1: ADC ON) If the bit is 0 and a 1 is written to it, the ADC module turns on. If the bit is 1 and a 1 is written to it, an ADC conversion starts.

- Bit de alinhamento



- Bit de calibração: o STM32F1xx pode fazer uma auto-calibração. Uma vez setado a calibração é feita e o hardware zera o bit ao termino da calibração. Interessante fazer esta operação a cada power up.
- Bit conversão contínua: seleciona entre uma conversão simples ou conversão contínua.
- Bit ADC ON: liga ou inicia o conversor AD. Primeira escrita 1 ativa o módulo. Segunda escrita 1 inicia conversão (single conversion). Se estiver usando conversão contínua (CONT = 1), basta ativar o ADC com ADON = 1 uma vez.

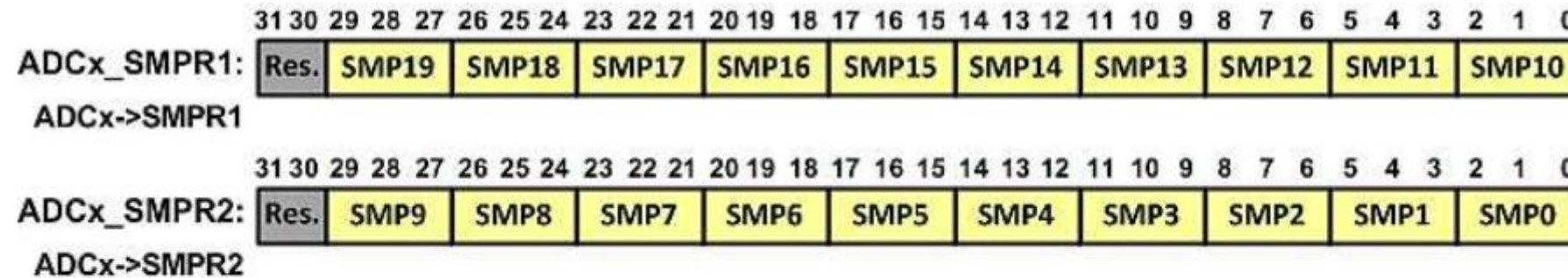
Registro de controle do multiplexador analógico

(define a **sequência de conversão dos canais do ADC** no STM32F103C8)



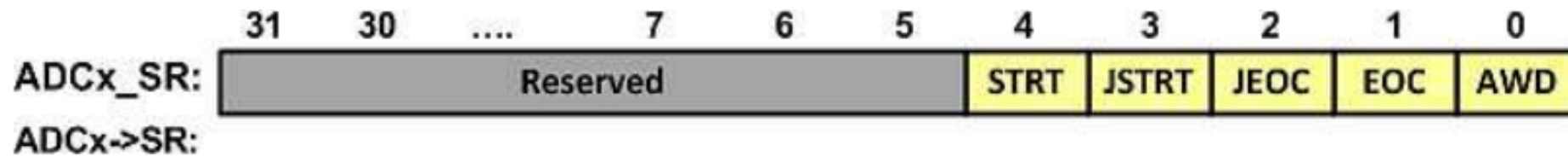
Field	Description
SQRn	Nth conversion in the sequence
L	Regular channel sequence Length Total number of conversions is L + 1. For more information see the manual.

Registro de tempos de amostragem



Field	Description			
SMPn	Channel n Sample time selection.			
	SMP	Sample time	SMP	Sample time
	000	1.5 ADC clock cycles	100	41.5 ADC clock cycles
	001	7.5 ADC clock cycles	101	55.5 ADC clock cycles
	010	13.5 ADC clock cycles	110	71.5 ADC clock cycles
	011	28.5 ADC clock cycles	111	239.5 ADC clock cycles

Registro de status da conversão



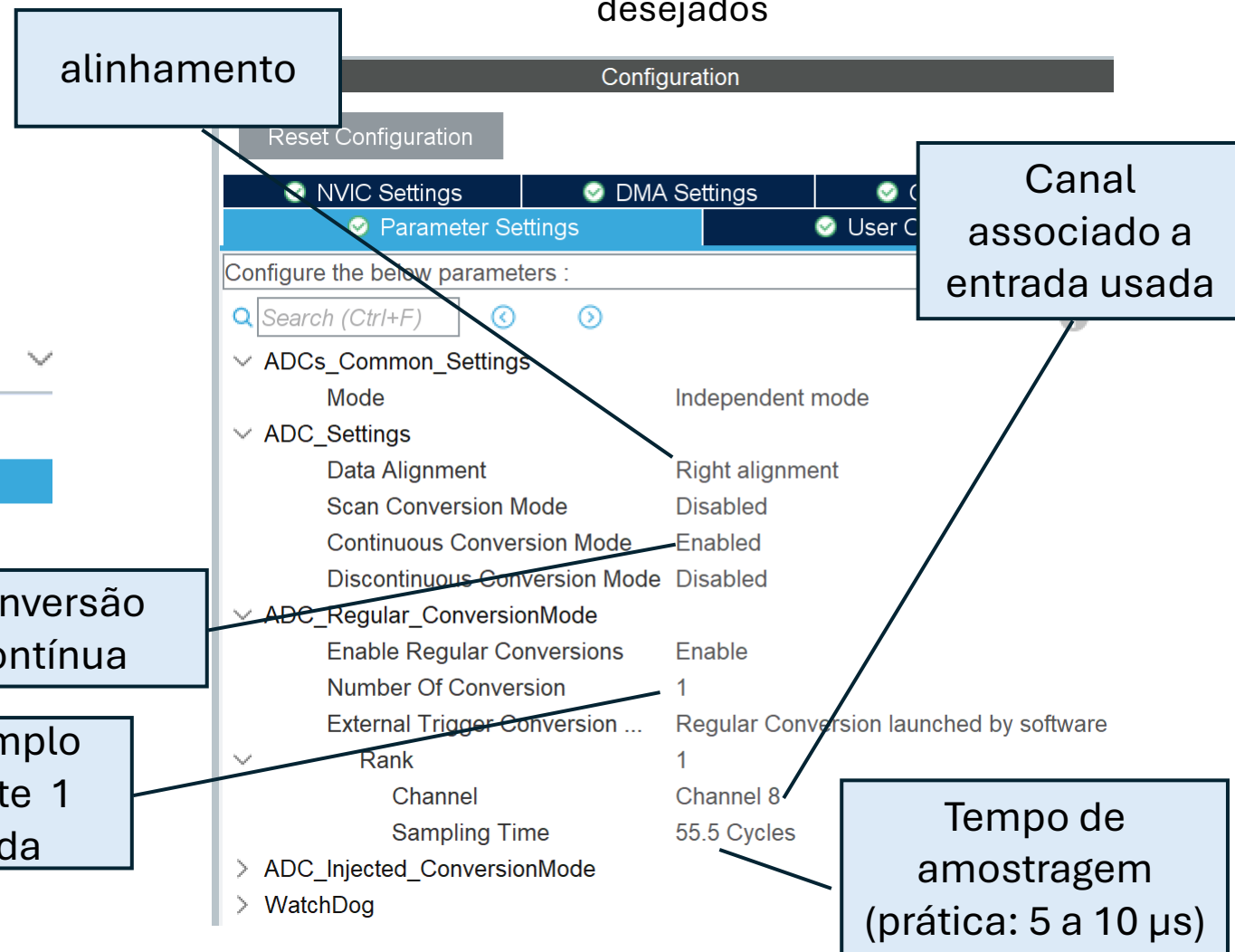
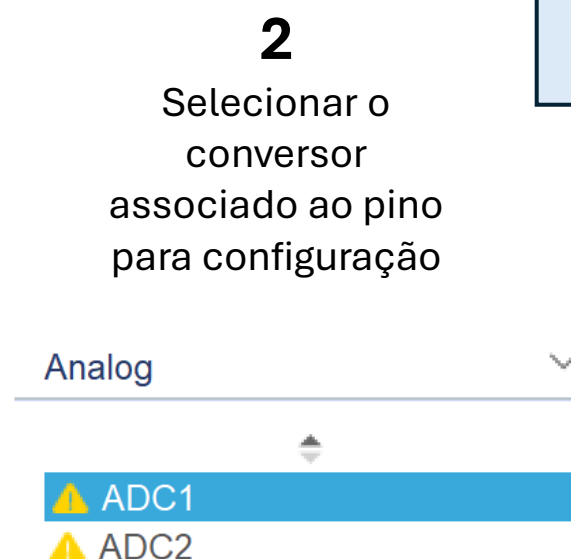
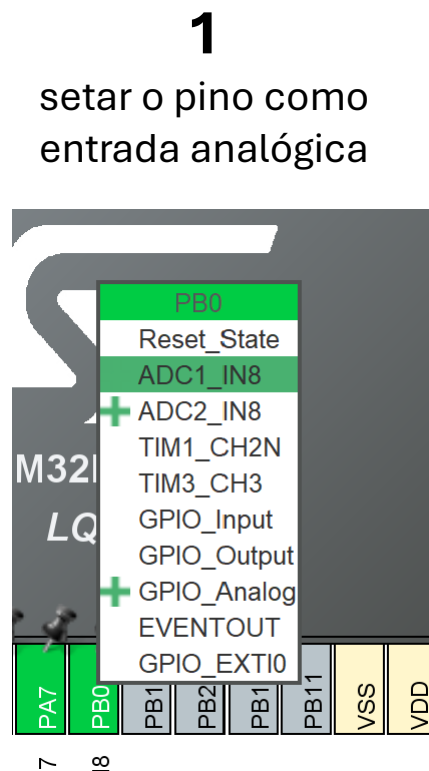
Field	Description
STRT	Regular channel Start flag
JSTRT	Injected channel Start flag
JEOC	Injected channel End Of Conversion
EOC	Regular channel End Of Conversion (0: conversion not complete, 1: complete)
AWD	Analog WatchDog flag

Exemplo de fluxo de conversão (simples)

1. Inicia conversão
2. 2. ADC converte
3. EOC = 1 (*HAL_ADC_PollForConversion(...)* pode ser chamada para aguardar fim da conversão)
4. Software lê ADC_DR

Configurações no CubeMX

(exemplo para o pino PB0: LDR na placa de testes)



Tempos de amostragem

Valores típicos recomendados

Situação do sinal	Sampling Time recomendado
Sinal de baixa impedância (op-amp buffer)	1.5 a 7.5 cycles
Sinal médio (potenciômetro)	28.5 a 55.5 cycles
Alta impedância (sensores passivos)	71.5 a 239.5 cycles

Exemplo de chamadas de acesso usando a biblioteca HAL

...

```
/* Infinite loop */
```

```
/* USER CODE BEGIN WHILE */
```

```
HAL_ADC_Start(&hadc1); // inicia a conversão
```

```
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY); // aguarda terminar a primeira
```

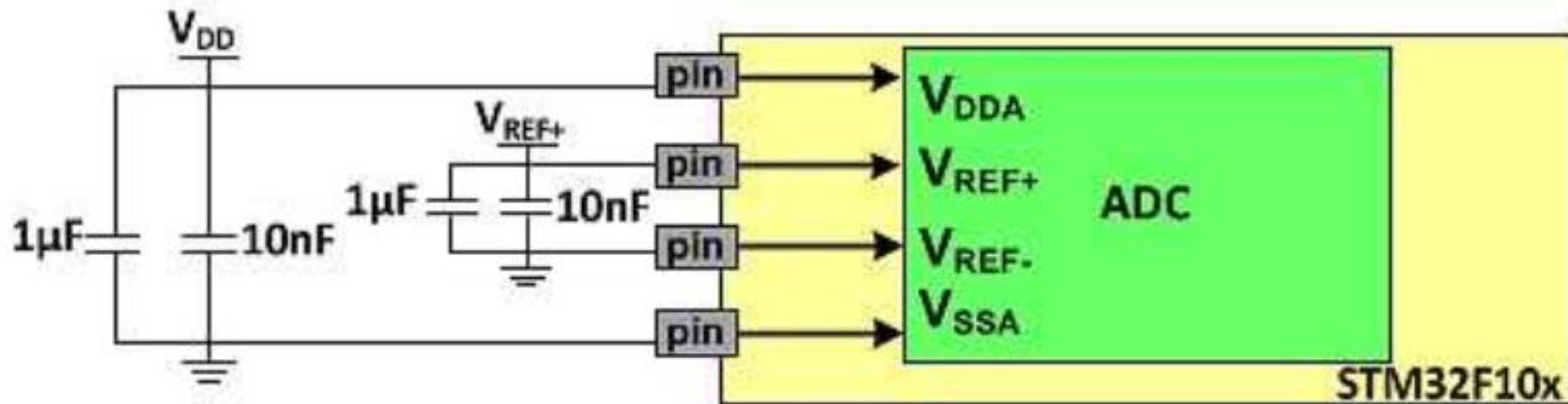
```
while (1)
```

```
{
```

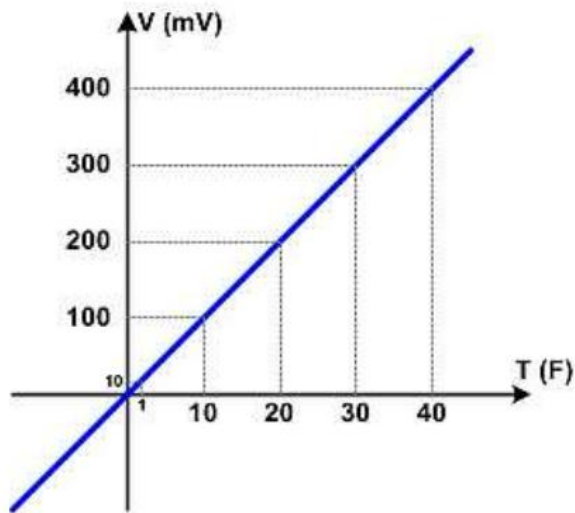
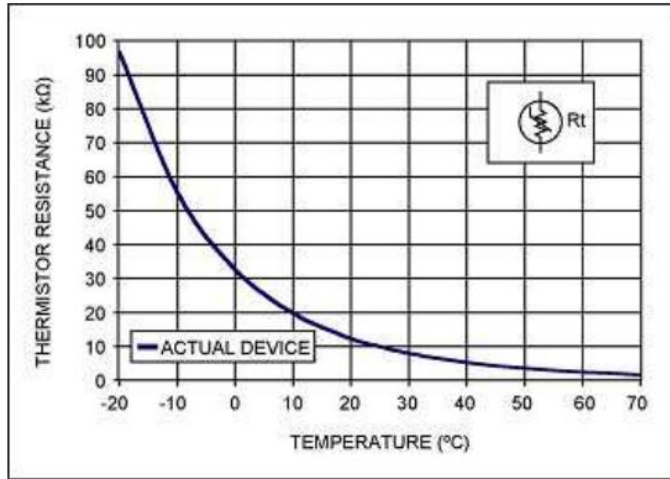
```
    contador = HAL_ADC_GetValue(&hadc1);
```

```
...
```

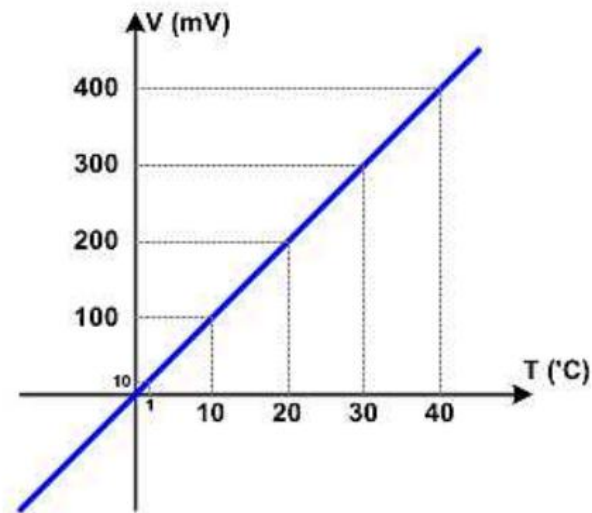
Filtros externos (minimizar ruidos)



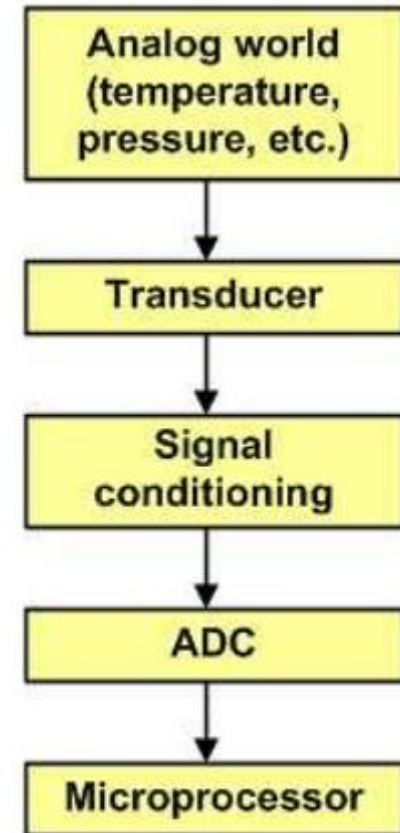
Condicionamento de sinal analógico



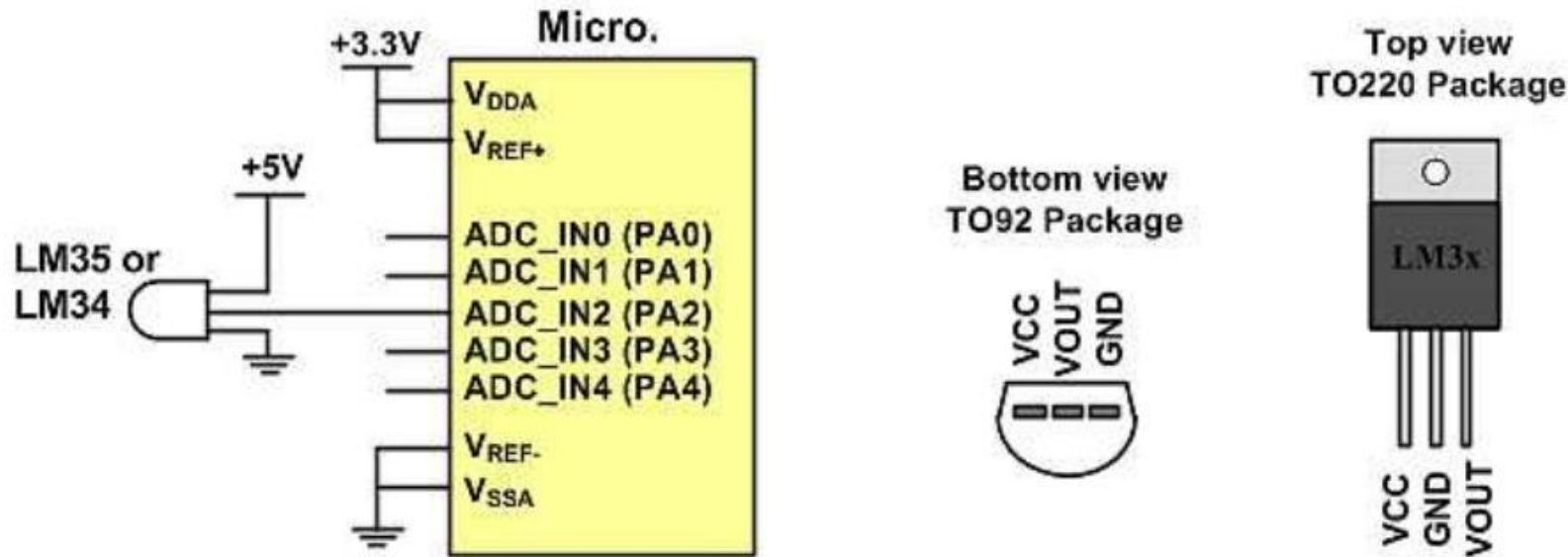
(a) LM34



(b) LM35



LM34/LM35: 10mV para cada grau de temperatura



- Como converter um valor lido pelo conversor AD (12bits) em temperatura no caso do LM34/LM35? (assumir $V_{ref} = 3.3V$)

The step size is $3.3/4096$ and for LM35/LM34, we have $V = 0.01 \times \text{temperature}$. So,

$$adcResult = \frac{V_{in}}{stepSize} = \frac{0.01 \times temp}{\frac{3.3}{4096}} \Rightarrow adcResult = \frac{4096 \times temp}{330} \Rightarrow temp = \frac{330 \times adcResult}{4096}$$

Usando as funções HAL

Verificar se a conversão terminou (polling):

```
HAL_StatusTypeDef HAL_ADC_PollForConversion(ADC_HandleTypeDef* hadc, uint32_t Timeout);
```

Ler o valor convertido:

```
uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef* hadc);
```

Exemplo completo

```
31  /* Private variables -----*/
32  ADC_HandleTypeDef hadc1;
33  UART_HandleTypeDef huart2;
34  char msg[30];
35  uint16_t rawValue;
36  float temp;
37
38  int main(void) {
39      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
40      HAL_Init();
41
42      /* Configure the system clock */
43      SystemClock_Config();
44
45      /* Initialize all configured peripherals */
46      MX_GPIO_Init();
47      MX_USART2_UART_Init();
48      MX_ADC1_Init();
```

```
49
50  /* Starts the ADC */
51  HAL_ADC_Start(&hadc1);
52
53  while (1) {
54      HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
55
56      rawValue = HAL_ADC_GetValue(&hadc1);
57      temp = ((float)rawValue) / 4095 * 3300;
58      temp = ((temp - 760.0) / 2.5) + 25;
59
60      sprintf(msg, "ADC rawValue: %hu\r\n", rawValue);
61      HAL_UART_Transmit(&huart2, (uint8_t*) msg, strlen(msg), HAL_MAX_DELAY);
62
63      sprintf(msg, "Temperature: %f\r\n", temp);
64      HAL_UART_Transmit(&huart2, (uint8_t*) msg, strlen(msg), HAL_MAX_DELAY);
65  }
66 }
67
```

```
68 static void MX_ADC1_Init(void) {
69     ADC_ChannelConfTypeDef sConfig = {0};
70
71     /** Configure the global features of the ADC (Clock, Resolution,
72     * Data Alignment and number of conversion) */
73     hadc1.Instance = ADC1;
74     hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV2;
75     hadc1.Init.Resolution = ADC_RESOLUTION_12B;
76     hadc1.Init.ScanConvMode = DISABLE;
77     hadc1.Init.ContinuousConvMode = ENABLE;
78     hadc1.Init.DiscontinuousConvMode = DISABLE;
79     hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
80     hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
81     hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
82     hadc1.Init.NbrOfConversion = 1;
83     hadc1.Init.DMAContinuousRequests = DISABLE;
84     hadc1.Init.EOCSelection = ADC_EOC_SEQ_CONV;
85     HAL_ADC_Init(&hadc1);
86
87     /** Configure for the selected ADC regular channel its corresponding
88     * rank in the sequencer and its sample time. */
89     sConfig.Channel = ADC_CHANNEL_TEMPSENSOR;
90     sConfig.Rank = 1;
91     sConfig.SamplingTime = ADC_SAMPLETIME_480CYCLES;
92     HAL_ADC_ConfigChannel(&hadc1, &sConfig);
93 }
```

Prática de laboratório

- Efetuar a leitura da entrada analógica associada ao LDR (ver circuito na documentação da placa de testes).